

```

        return False # stays in the window

    ❹ def draw(self):
        self.image.draw()

    ❺ def collide(self, playerRect):
        collidedWithPlayer = self.image.overlaps(playerRect)
        return collidedWithPlayer

```

Listing 16-8: The `Baddie` class

We load the image of the Baddie as a class variable ❶ so the single image is shared by *all* Baddies.

The `__init__()` method ❷ chooses a random size for each new Baddie, so the user sees differently sized Baddies. It chooses a random x-coordinate and a y-coordinate that will place the image just above the window. It then creates an `Image` object and scales the image down to the selected size ❸. Lastly, it chooses a random speed.

The Baddie manager, the code for which I'll show in a moment, calls the `update()` method ❹ in every frame: the code here moves the location of the Baddie down by the number of pixels that represents its speed. If the Baddie has moved off the bottom of the game area, we return `True` to say this Baddie is ready to be removed. Otherwise, we return `False` to tell the Baddie manager to leave this Baddie in the window.

The `draw()` method ❺ draws the Baddie at its new location.

The `collide()` method ❻ checks to see if the Player and the Baddie intersect.

The `BaddieMgr` class, shown in [Listing 16-9](#), creates and manages a list of Baddie objects; this is a classic example of an object manager object.

```

# BaddieMgr class
class BaddieMgr():
    ADD_NEW_BADDIE_RATE = 8 # how often to add a new Baddie

    ❶ def __init__(self, window):
        self.window = window
        self.reset()

```